

Final Report

Student Name: Xu Chen

Student ID: 2257453

1. Understanding of the Scrum Framework

Scrum is an agile framework designed to deliver value iteratively through time-boxed cycles called Sprints (typically 2-4 weeks). Teams prioritize Product Backlog Items (PBIs)—user-centric requirements broken into actionable tasks—and select high-priority PBIs for each Sprint. Key roles include:

Product Owner: Defines PBIs and prioritizes the backlog.

Scrum Master: Facilitates Scrum processes and removes blockers.

Development Team: Implements PBIs and delivers increments.

Scrum events include Sprint Planning (selecting PBIs), Daily Scrum (progress sync), Sprint Review (demo deliverables), and Retrospective (process improvement). PBIs bridge user needs with technical execution, ensuring incremental value delivery.

Word Count: 91/100

2. Pre-Groomed PBI (7.1 to 7.5)

2.1 PBI 7.1: Admin Review of User-Uploaded Resources

User Story: As an admin, I want to review music resources uploaded by users so that I can ensure content compliance.

Acceptance Criteria:

- 1 Admin can view all user-uploaded music resources.

GIVEN a list of user-uploaded music resources exists,

WHEN the admin accesses the review page,

THEN the admin can view all resources sorted by upload date (newest first).

- 2 Admin can mark resources as "reviewed" or "unreviewed".

GIVEN a music resource is displayed on the review page,

WHEN the admin selects the "reviewed" or "unreviewed" option,

THEN the system updates the resource status and persists the change immediately.

- 3 System logs review time and reviewer.

GIVEN a music resource is reviewed by the admin,
WHEN the admin completes the review,
THEN the system logs the review time, reviewer's ID, and action type.

- 4 Performance: Load time for large datasets.

GIVEN there are 1,000+ music resources in the system,
WHEN the admin opens the review page,
THEN the list loads within 2 seconds.

- 5 Error Handling: Failed review status update.

GIVEN the admin attempts to mark a resource as "reviewed",
WHEN the system fails to update the status due to a network error,
THEN an error message is displayed with a "Retry" button.

- 6 Data Validation: Prevent duplicate reviews.

GIVEN a music resource is already marked as "reviewed",
WHEN the admin tries to review it again,
THEN the system displays a warning: "This resource has already been reviewed."

- 7 Accessibility: Screen reader compatibility.

GIVEN the admin uses a screen reader,
WHEN navigating the review page,
THEN all resource details and actions are announced clearly.

2.2 PBI 7.2: Block Unauthorized/Prohibited Content

User Story: As an admin, I want to block unauthorized or prohibited music resources so that I can maintain platform content quality.

Acceptance Criteria:

- 1 Admin can block individual resources.

GIVEN a music resource is displayed on the admin dashboard,
WHEN the admin marks it as "blocked",
THEN the system updates the resource status to "blocked".

- 2 Blocked resources are no longer visible to users.

GIVEN a music resource is marked as "blocked",

WHEN a user tries to access the resource via search or direct link,
THEN the system returns a "404 Not Found" error.

- 3 System notifies the uploader of the blocked resource.

GIVEN a music resource is blocked by the admin,
WHEN the action is completed,
THEN the system sends an email to the uploader with the reason for blocking.

- 4 Performance: Bulk blocking efficiency.

GIVEN the admin selects 50 resources to block,
WHEN the admin confirms the action,
THEN all resources are blocked within 5 seconds.

- 5 Error Handling: Failed blocking action.

GIVEN the admin attempts to block a resource,
WHEN the system fails due to a database error,
THEN the system rolls back the action and displays: "Failed to block resource.
Please try again."

- 6 Data Integrity: Archive blocked resources securely.

GIVEN a resource is blocked,
WHEN the action is completed,
THEN the resource is moved to a read-only archive with restricted access.

- 7 Accessibility: Keyboard navigation for blocking.

GIVEN the admin uses keyboard navigation,
WHEN interacting with the "Block" button,
THEN the action can be triggered via the Enter key.

2.3 PBI 7.3: Lock and Unlock User Accounts

User Story: As an admin, I want to lock and unlock user accounts so that I can manage violating users.

Acceptance Criteria:

- 1 Admin can lock user accounts.

GIVEN a user account is displayed on the admin dashboard,

WHEN the admin selects the "Lock" option and provides a reason,
THEN the system locks the account and logs the action.

- 2 Locked users cannot log in or upload resources.

GIVEN a user account is locked,
WHEN the user attempts to log in,
THEN the system displays: "Your account is locked. Contact support."

- 3 Admin can unlock accounts, restoring user permissions.

GIVEN a locked user account is displayed on the admin dashboard,
WHEN the admin selects the "Unlock" option,
THEN the system restores permissions and notifies the user via email.

- 4 Performance: Account lock/unlock latency.

GIVEN the admin initiates a lock/unlock action,
WHEN the action is confirmed,
THEN the system applies the change within 1 second.

- 5 Error Handling: Concurrent account modification.

GIVEN two admins try to lock the same account simultaneously,
WHEN conflicts occur,
THEN the system prevents duplicate actions and displays: "This account is being modified by another admin."

- 6 Audit Trail: Log lock/unlock history.

GIVEN an admin locks or unlocks an account,
WHEN the action is completed,
THEN the system records the timestamp, admin ID, and reason in an audit log.
(modified after peer feedback)

- 7 Security: Two-factor authentication for critical actions.

GIVEN the admin attempts to unlock a high-risk account,
WHEN the action is triggered,
THEN the system requires OTP verification before proceeding.

2.4 PBI 7.4: View All Music Resources

User Story: As an admin, I want to view all music resources so that I can

comprehensively manage platform content.

Acceptance Criteria:

1 Paginated display of resources

GIVEN 5,000+ music resources exist in the system,
WHEN the admin accesses the resource management page,
THEN the system displays the first 50 resources with pagination controls.

2 Display metadata and tooltip details

GIVEN a music resource is listed,
WHEN the admin hovers over a row,
THEN a tip box shows additional details (e.g., duration, bit rate).

3 Admin can filter resources by category, upload date, or license type.

GIVEN the admin selects "Rock" as the category and "Last 7 days" as the date range,
WHEN the filter is applied,
THEN the list updates to show only matching resources.

4 Performance: Filter response time.

GIVEN the admin applies a complex filter (e.g., "Category=Pop + License=CC-BY"),
WHEN the filter is submitted,
THEN the results load within 1.5 seconds.

5 Error Handling: Handle invalid filter inputs

GIVEN the admin enters an invalid date range (e.g., "3000-01-01"),
WHEN the filter is applied,
THEN the system displays: "Invalid date range. Please select a valid range."

6 Export resources as CSV with data limits

GIVEN the admin clicks the "Export" button,
WHEN the action is confirmed,
THEN the system generates a CSV file containing up to 10,000 visible resources per export. If more resources exist, the system should prompt the admin to apply filters or split the export into multiple files.(modified after peer feedback)

7 Accessibility: Keyboard navigation for pagination.

GIVEN the admin uses keyboard navigation,
WHEN interacting with pagination controls,
THEN the "Next" and "Previous" buttons can be triggered via arrow keys.

2.5 PBI 7.5: Manage User Accounts

User Story: As an admin, I want to manage user accounts so that I can maintain the platform's user base.

Acceptance Criteria:

1 View and sort accounts by role.

GIVEN 1,000+ user accounts exist,
WHEN the admin sorts by "Role" (Admin/User),
THEN the list groups accounts by role with collapsible sections.

2 Edit user roles securely.

GIVEN a user account is selected,
WHEN the admin edits the role to "Moderator",
THEN the system validates permissions against a predefined policy matrix.

3 Delete accounts with data anonymization

GIVEN a user account is marked for deletion,
WHEN the admin confirms the action,
THEN the system anonymous personal data (e.g., email → "user123@deleted.com").

4 Bulk deletion efficiency

GIVEN the admin selects 100 users for deletion,
WHEN the action is confirmed,
THEN the system deletes all accounts within 10 seconds and creates a backup of the deleted data, allowing recovery within 30 days. (modified after peer feedback)

5 Error Handling: Concurrent edits.

GIVEN two admins edit the same user's role simultaneously,
WHEN conflicts occur,
THEN the system prevents overwrites and displays: "This account is being

modified by another admin."

6 Audit Trail: Track account changes.

GIVEN an admin edits or deletes a user account,
WHEN the action is completed,
THEN the system logs the admin ID, timestamp, and modified fields.

7 Security: Session timeout for sensitive actions.

GIVEN the admin is inactive for 5 minutes while editing a user account,
WHEN they attempt to save changes,
THEN the system requires reauthentication, and unsaved changes are retained for 10 minutes to allow the admin to resume editing after reauthentication.
(modified after peer feedback)

3. Collaboration and Reflection

3.1 Peer Review and Feedback:

I actively engaged in peer reviews by leading team discussions to align on evaluation criteria. During meetings, we debated ambiguities, highlighted missing security scenarios and suggested error messages like "invalid token." We reviewed PBIs line-by-line, voted on priority fixes, and documented agreements. My key contributions included standardizing validation rules and advocating for edge cases . The following are suggestions for improvement :

PBI 1:

PBI 1.4: In addition to the 24h expiration, if a user attempts to use a reset link that has been tampered with or expired, the system will display an "Invalid Token Error".

PBI 2:

PBI 2.1: Add avatar format/size validation (e.g., "max 5MB, PNG/JPG").

PBI 2.5: Add profile export option (e.g., "download as PDF").

PBI 3 :

PBI 3.1: Add category name character limit (e.g., "max 50 characters").

PBI 3.3: Add resource transfer option (e.g., "move files to another category").

PBI 3.4: Enable sorting options (e.g., "by name/creation date").

PBI 4 :

PBI 4.1: Add upload retry limit (e.g., "max 3 attempts").

PBI 4.3: Add auto-block for repeated violations (e.g., "3 times").

PBI 4.5: Track re-upload attempts (e.g., "max 2 revisions").

PBI 5 :

PBI 5.3: Auto-delete old history (e.g., "after 90 days").

PBI 6 :

PBI 6.1: Add playlist creation (e.g., "queue management").

PBI 8 :

PBI 8.1: Add feedback priority tags (e.g., "Urgent/Normal").

PBI 8.2: Enable article voting (e.g., "mark helpful").

Word count: 227 / 250

3.2 Reflection on Learning

Through the process, I **learned** the importance of clear user stories and precise acceptance criteria. We met challenges about aligning priorities. Through structured discussions and voting, we learned to balance competing priorities. Feedback from peers highlighted critical gaps and I **improved by adding detailed scenarios (export data limits in PBI 7.4 and bulk deletion recovery in PBI 7.5) and refining error handling (PBI 7.5 session timeout with unsaved changes retention)**. Personally, I **grew** in my ability to communicate technical requirements clearly. This experience **enhanced my ability** to write testable PBIs and collaborate effectively, ensuring deliverables align with stakeholder needs.

Word Count: 99/100